coverage.py vs. pytest-cov: How They Relate and When to Use Which

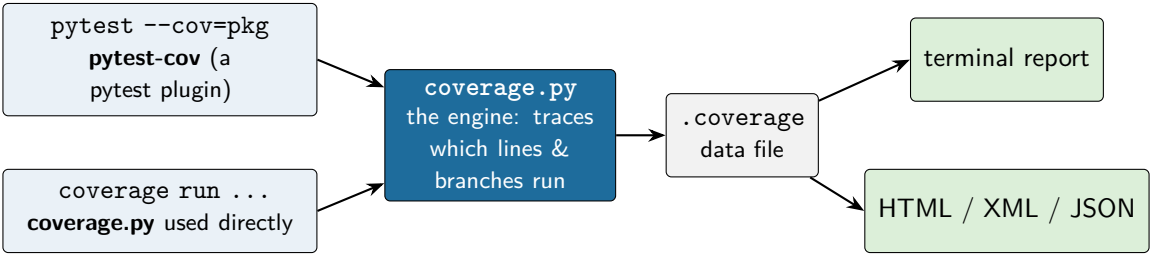
SE 413/513: Software Testing

Dr. Kamrul Hasan

Grand Valley State University

Fall 2026

One Engine, One Plugin



- `coverage.py` measures *any* Python program: a test run, a script, a CLI.
- `pytest-cov` switches `coverage.py` on and off *inside* `pytest` and prints the report at the end.

Key Idea

Not competitors: `pytest-cov` **depends on** `coverage.py`. Same engine ⇒ same numbers.

Our Tiny Example

`tempconv/temps.py`

```python
def to_fahrenheit(c):
    return c * 9 / 5 + 32


def describe(c):
    if c < 0:
        return "freezing"
    elif c < 25:
        return "mild"
    return "hot"
```

`tests/test_temps.py`

```python
from tempconv.temps import to_fahrenheit, describe

def test_boiling():
    assert to_fahrenheit(100) == 212

def test_mild():
    assert describe(20) == "mild"
```

`report.py` — a command-line tool, **not** a test:

```python
for arg in sys.argv[1:]:
    c = float(arg)
    f = to_fahrenheit(c)
    print(f"{c} C = {f} F ({describe(c)})")
```

Setup: `pip install pytest coverage pytest-cov` (installing `pytest-cov` pulls in `coverage` anyway).

Same Measurement, Two Ways

`pytest-cov`: one command

```
pytest --cov=tempconv --cov-branch \
       --cov-report=term-missing
```

`coverage.py`: measure, then report

```
coverage run --branch --source=tempconv
    -m pytest
coverage report -m
```

Both print:

```
Name                    Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------
tempconv/temps.py           8      2      4      2    67%   7, 10
```

pytest-cov	coverage.py	pytest-cov	coverage.py
`--cov=PKG`	`run --source=PKG`	`--cov-report=html`	`coverage html`
`--cov-branch`	`run --branch`	`--cov-report=xml`	`coverage xml`
`--cov-report=term-missing`	`report -m`	`--cov-fail-under=N`	`report --fail-under=N`

They Share a Data File and a Config

1. One data file. After a `pytest-cov` run:

```
pytest --cov=tempconv --cov-branch
ls -a                # .coverage is here
coverage report -m   # no re-run
coverage html        # same data
```

```
tempconv/temps.py   8   2   4   2   67%   7, 10
```

Measure once, report as many ways as you like.

2. One config. In `pyproject.toml`:

```
[tool.coverage.run]
branch = true
source = ["tempconv"]

[tool.coverage.report]
show_missing = true
fail_under = 80
```

Now `pytest --cov` and `coverage run -m pytest` need no extra flags — and every teammate and CI job gets the **same** numbers.

Where They Differ

	`pytest-cov`	`coverage.py` **directly**
What it can measure	only a `pytest` run	any Python process
Commands	one (`pytest --cov`)	two or more (`run`, then `report`/`html`...)
Parallel tests (`pytest-xdist`)	handled automatically	needs extra setup
Combining separate runs	automatic for its own workers	`--append`, `combine`
Re-report without re-running	—	`coverage report`/`html` anytime
Non-pytest code (scripts, `unittest`)	no	yes
Typical home	everyday testing, CI	investigation, special cases

When to Reach for `coverage.py` Directly

A. Code that `pytest` doesn't run

```
coverage erase
coverage run --branch --source=tempconv \
         report.py -5 30
coverage report -m
```

```
-5.0 C = 23.0 F (freezing)
30.0 C = 86.0 F (hot)
tempconv/temps.py    8    1    4    1    83%    9
```

Also works for `unittest`, a module (`-m pkg`), or a manual exploratory session.

B. Adding runs together

```
coverage run --append --branch \
         --source=tempconv -m pytest
coverage report -m
```

```
tempconv/temps.py    8    0    4    0    100%
```

Run	Cover
tests only	67% (missing 7, 10)
script only	83% (missing 9)
script + tests	**100%**

C. Re-reporting old data (`coverage html`, `coverage report --skip-covered`) without paying for another test run.

When to Use `pytest-cov`: Everyday Tests, Parallel Runs, CI

Parallel tests with `pytest-xdist` (`-n 2`):

```
pytest -n 2 --cov=tempconv --cov-branch
```

```
tempconv/temps.py    8    2    4    2    67%
2 passed in 0.50s
```

```
coverage run --branch --source=tempconv \
         -m pytest -n 2
coverage report
```

```
CoverageWarning: No data was collected.
tempconv/temps.py    8    8    4    0     0%
```

`coverage run` traced only the main process; the tests ran in **workers** it never saw. `pytest-cov` measures each worker and combines the data.

A coverage gate in CI:

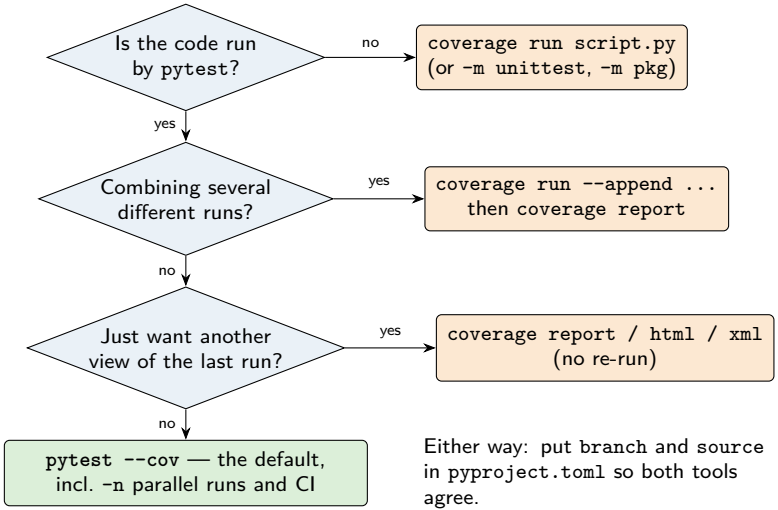
```
pytest --cov=tempconv --cov-branch \
       --cov-fail-under=90
```

```
FAIL Required test coverage of 90%
  not reached. Total: 66.67%
2 passed in 0.02s
```

Tests pass, but the **command fails** (exit 1) $\Rightarrow$ CI build fails.

```
# GitHub Actions step
- run: pytest --cov --cov-branch
       --cov-report=xml
       --cov-fail-under=80
```

Which One Should I Use?



Either way: put `branch` and `source` in `pyproject.toml` so both tools agree.

Pitfalls and Takeaways

Forget `--source` and the number lies:

```
$ coverage run -m pytest && coverage report
tempconv/temps.py          8     2    75%
tests/test_temps.py        5     0   100%
TOTAL                     13     2    85%
```

Test files are always ∼100% run — they inflate the total.

Two More

Stale data: `--append` keeps old results — `coverage erase` before a fresh measurement.

Wrong target: `--cov=` names the code *measured*, not the tests. If it's never imported you get only a warning (*"No data was collected"*) — and green tests.

Key Idea

1. `coverage.py` **is the engine**; `pytest-cov` is a plugin that drives it from `pytest`.
2. Same engine, same `.coverage` file, same config ⇒ **same numbers**.
3. **Default to** `pytest --cov`, especially for parallel runs and CI.
4. **Drop to** `coverage` for non-pytest code, combining runs, or re-reporting.
5. Always measure **branches** and only **your own code**.